

UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS
Facultad de Ciencias Puras · Carrera de Ingeniería Informática
Potosí — Bolivia

INF 220 — PROGRAMACIÓN ORIENTADA A OBJETOS Y ESTRUCTURAS DE DATOS

GUÍA DE LABORATORIO N.º 6

Colecciones del framework de Java

Unidad	Unidad 1 — Programación Orientada a Objetos (guía de cierre)
Tema	Arreglos vs. colecciones · genéricos · ArrayList · HashMap · HashSet · recorridos e iteradores
Requisito	Guía de Laboratorio N.º 5 (cadenas y manejo de texto)
Stack	JDK Temurin 25 (LTS) · Visual Studio Code · Extension Pack for Java
Duración	1 sesión de laboratorio
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán

1 Objetivos

► Objetivo general

Emplear las principales colecciones del Java Collections Framework —ArrayList, HashMap y HashSet— para almacenar y manipular conjuntos de datos, recorriéndolas con distintas técnicas.

► Objetivos específicos

- Diferenciar un arreglo de tamaño fijo de una colección dinámica.
- Usar ArrayList para listas cuyo tamaño cambia en tiempo de ejecución.
- Usar HashMap para almacenar y consultar pares clave-valor.
- Usar HashSet para trabajar con conjuntos de elementos únicos.
- Recorrer colecciones con for-each y con iteradores.

2 Requisitos previos

Software	JDK Temurin 25 + Visual Studio Code con Extension Pack for Java.
Conocimientos	Clases y objetos, arreglos, métodos, control de flujo y cadenas (Guías 1 a 5).

3 Marco teórico

► De los arreglos a las colecciones

Un arreglo (`int[]`) tiene un tamaño **fijo** que se define al crearlo y no puede cambiar. Cuando no sabemos de antemano cuántos elementos habrá, necesitamos estructuras que crezcan y se reduzcan solas: las **colecciones** del Java Collections Framework.

DEFINICIÓN · Java Collections Framework

Es el conjunto de clases e interfaces estándar de Java para manejar grupos de objetos. Las tres más usadas son `ArrayList` (listas), `HashMap` (pares clave-valor) y `HashSet` (conjuntos sin duplicados).

► Genéricos: el tipo entre < >

Al declarar una colección se indica entre < > el tipo de elementos que guardará, por ejemplo `ArrayList<String>`. Así el compilador garantiza que solo se guarden elementos de ese tipo.

IMPORTANTE · Colecciones y tipos envoltorio

Las colecciones solo almacenan objetos, no primitivos. En lugar de `int` o `double` se usan sus clases envoltorio: `Integer` y `Double`. Java convierte entre ambos automáticamente (autoboxing).

► Las tres colecciones esenciales

<code>ArrayList<T></code>	Lista ordenada y dinámica; permite duplicados y acceso por índice. Métodos: <code>add</code> , <code>get</code> , <code>set</code> , <code>remove</code> , <code>size</code> , <code>contains</code> .
<code>HashMap<K,V></code>	Asocia una clave única a un valor. Métodos: <code>put</code> , <code>get</code> , <code>containsKey</code> , <code>keySet</code> , <code>remove</code> .
<code>HashSet<T></code>	Conjunto sin elementos repetidos y sin orden garantizado. Métodos: <code>add</code> , <code>contains</code> , <code>remove</code> , <code>size</code> .

► Recorrer una colección

- **for-each** — la forma más simple: `for (String s : lista) { ... }`.
- **Iterator** — permite recorrer y *eliminar* elementos de forma segura durante el recorrido con `it.remove()`.

4 Desarrollo de la práctica

Trabaja en una carpeta `Lab06`. Recuerda escribir los `import` al inicio de cada archivo.

► Actividad 1 — Del arreglo al ArrayList

Crea una lista dinámica de nombres y agrégle elementos:

```
import java.util.ArrayList;

ArrayList<String> nombres = new ArrayList<>();
nombres.add("Ana");
nombres.add("Luis");
nombres.add("Marta");

System.out.println("Cantidad: " + nombres.size());
System.out.println("Primero: " + nombres.get(0));
```

► Actividad 2 — Operaciones con ArrayList

Modifica, elimina y consulta elementos de la lista:

```
nombres.set(1, "Luisa");           // reemplaza el elemento en la posición 1
nombres.remove("Marta");          // elimina por valor

System.out.println(nombres.contains("Ana")); // true
System.out.println("Tamaño: " + nombres.size());
System.out.println(nombres);        // [Ana, Luisa]
```

► Actividad 3 — Recorrer con for-each e iterador

Recorre la lista de las dos maneras:

```
// 1) for-each
for (String n : nombres) {
    System.out.println(n);
}

// 2) con un iterador
import java.util.Iterator;
Iterator<String> it = nombres.iterator();
while (it.hasNext()) {
    System.out.println(it.next());
}
```

NOTA · Eliminar mientras se recorre

Si necesitas borrar elementos durante el recorrido, hazlo con `it.remove()` del iterador; eliminar directamente de la lista dentro de un `for-each` provoca un `ConcurrentModificationException`.

► Actividad 4 — HashMap: pares clave-valor

Asocia nombres con edades y consúltalos por su clave:

```
import java.util.HashMap;

HashMap<String, Integer> edades = new HashMap<>();
edades.put("Ana", 20);
edades.put("Luis", 22);

System.out.println("Edad de Ana: " + edades.get("Ana"));
System.out.println(edades.containsKey("Luis"));

for (String clave : edades.keySet()) {
    System.out.println(clave + " -> " + edades.get(clave));
}
```

► Actividad 5 — HashSet: elementos únicos

Comprueba que un conjunto no admite duplicados:

```
import java.util.HashSet;

HashSet<String> colores = new HashSet<>();
```

```
colores.add("rojo");
colores.add("verde");
colores.add("rojo");           // duplicado: se ignora

System.out.println("Tamaño: " + colores.size()); // 2
System.out.println(colores.contains("verde"));   // true
```

► Actividad 6 — Una colección de objetos

Reutiliza la clase Estudiante (con su constructor de la Guía 3) y guárdala en una lista:

```
ArrayList<Estudiante> curso = new ArrayList<>();
curso.add(new Estudiante("Ana", 20, 78.5));
curso.add(new Estudiante("Luis", 22, 45.0));
curso.add(new Estudiante("Marta", 19, 90.0));

for (Estudiante e : curso) {
    e.mostrarDatos();
}
```

EJEMPLO · El poder de combinar

Una colección de objetos une todo lo aprendido en la unidad: clases, encapsulamiento, métodos y estructuras dinámicas trabajando juntas. Es la base de lo que verás en la Unidad 4 con las estructuras de datos.

5 Bibliografía y recursos

- Deitel, P. & Deitel, H. — Cómo programar en Java. Pearson.
- Sierra, K. & Bates, B. — Head First Java. O'Reilly.
- Collections Framework (tutorial oficial): <https://docs.oracle.com/javase/tutorial/collections/>